

SecondSelf — Your Personal AI Second Brain

National Hackathon PRD v3 — Multi-Modal RAG + Knowledge Graph + Persistent Memory

1. One-line thesis

SecondSelf turns scattered notes, files, links, images, and voice memos into a self-organizing, queryable, visual second brain that answers questions from your own knowledge and remembers every conversation.

2. Problem

Every notes app fails the same way: capture is easy, retrieval is broken. You save PDFs, screenshots, voice memos, spreadsheets, and links — then never find them again. Information goes in, but knowledge never compounds.

3. Insight

The bottleneck is not capture. It is **normalization + semantic linking + memory**. If every capture becomes the same internal record, AI can classify it, embed it, link it, graph it, and answer from it across formats.

4. Solution

A full-stack web app with six core capabilities:

1. **Capture anything** — note, link, PDF, DOCX, TXT/MD, image, audio, CSV/XLSX.

2. **Normalize every capture** — original file + extracted text + metadata + timestamp + unique ID.
3. **Auto-classify** — PARA category, tags, one-line summary, confidence.
4. **Auto-link** — embeddings + cosine similarity → related notes linked across formats.
5. **Visualize** — live interactive force-directed graph with node types, hover previews, drag, zoom.
6. **Ask + remember** — RAG-based Q&A with citations; every session persisted and reopenable.

5. Why this is technically hard

- Multi-modal ingestion: PDF text, DOCX, OCR, ASR, CSV summarization, link extraction.
- Cross-format semantic linking: a PDF, an image, and an audio transcript must land in the same embedding space.
- Long-document retrieval: chunking, overlap, parent-document retrieval, citation mapping.
- RAG faithfulness: answers must cite retrieved notes, not hallucinate.
- Persistent conversational memory: sessions, retrieved context, follow-up context.
- Graph UX: real-time rendering from backend JSON with meaningful node/edge semantics.

6. System architecture

```

Frontend: React/Next.js + vis-network/Cytoscape.js
  Capture | Graph | Ask | History
    ↓ REST/JSON
Backend: FastAPI
  /capture /classify /link /graph /ask /history
    ↓
Ingestion Layer
  parsers/: pdf, docx, image_ocr, audio_transcribe, csv, link
    ↓
Storage Layer
  raw/          original file + extracted text + metadata
  wiki/         classified + linked notes
  SQLite        notes, links, embeddings metadata, chat sessions
  Vector Index  FAISS/Chroma + SQLite FTS5 for hybrid search
    ↓
AI Layer
  LLM: Groq/Llama 3 + fallback Gemini/OpenAI
  Embeddings: sentence-transformers (all-MiniLM-L6-v2 or bge-small)
  OCR: Tesseract/PaddleOCR
  ASR: faster-whisper/Whisper.cpp

```

7. AI/ML pipeline details

7.1 Capture + extraction

- PDF: PyMuPDF; fallback OCR for scanned PDFs.
- DOCX: python-docx.
- Image: OCR + optional vision caption/description.
- Audio: faster-whisper transcription.
- CSV/XLSX: pandas/openpyxl → schema + row/column summary.
- Link: trafilatura/BeautifulSoup → main content + title.
- Every item normalized to:

```
{
  "id": "uuid",
  "timestamp": "ISO",
  "source_type": "pdf|docx|image|audio|csv|link|note",
  "raw_path": "raw/...",
  "extracted_text": "...",
  "metadata": {}
}
```

7.2 Classification

Send extracted text to LLM with strict JSON schema:

```
{
  "category": "Projects|Areas|Resources|Archives",
  "tags": ["..."],
  "summary": "one line",
  "confidence": 0.0
}
```

Fallback: keyword/regex classifier if LLM fails.

7.3 Embeddings + auto-linking

- Chunk long text: 500–800 tokens, 15% overlap.
- Embed chunks with sentence-transformers.
- Store in FAISS/Chroma.
- Link new note to existing notes if cosine similarity > threshold.
- Avoid hub explosion: cap top-k links per note, dedupe edges.

7.4 RAG Q&A

- Hybrid retrieval: BM25/FTS5 + vector search.
- Optional reranker: cross-encoder or LLM rerank.
- LLM prompt: "Answer only from these notes. Cite note IDs."
- Return: answer + cited notes + retrieved context.

7.5 Chat history

SQLite tables:

- sessions(id, title, created_at, updated_at)
- messages(id, session_id, role, content, created_at)
- retrievals(id, message_id, note_id, chunk_id, score)

Endpoints:

- POST /ask
- GET /history
- GET /history/{session_id}
- POST /history/{session_id}/continue

8. Feature breakdown

Feature	Must-have	Technical proof
Capture	Note, link, PDF, image, audio, CSV	Multi-format parsers
Normalize	Timestamp, UUID, source type, raw file, text	Unified schema
Classify	PARA + tags + summary	Structured LLM output
Link	Auto-link related notes	Embeddings + similarity
Graph	Interactive force graph	vis-network/Cytoscape
Ask	RAG with citations	Hybrid retrieval + LLM
History	Save/reopen/continue chats	SQLite session memory
Deploy	Public URL	Vercel/Netlify + Render/Railway

9. Week-by-week milestones

Week 1 — The Archivist

Ship multi-format capture API + upload UI.

Acceptance: 10+ real items, 3+ formats, normalized schema.

Week 2 — The Librarian

Ship PARA classification + embeddings + auto-linking.

Acceptance: 15+ real items in `wiki/` , cross-format links.

Week 3 — The Cartographer

Ship graph JSON endpoint + interactive graph in app.

Acceptance: hover, drag, zoom, source-type styling.

Week 4 — The Oracle

Ship RAG `ask()` , chat history, polished UI, deployment.

Acceptance: public URL, cross-modal Q&A, saved sessions.

10. Evaluation plan

Judges love measurable AI. Add this:

- **Extraction accuracy:** 10 files, manual check of extracted text.
- **Classification:** 30 notes, precision/recall/F1 for PARA.
- **Retrieval:** 20 Q/A pairs, Recall@5 and MRR.
- **Answer faithfulness:** LLM-as-judge or manual 1–5 score.
- **Citation accuracy:** % answers citing correct notes.
- **Latency:** p95 `/ask` under 8s.
- **Cost:** under \$0.01/query using free tiers.

- **Graph:** 100+ nodes load under 2s.

11. 3-minute demo script

1. **0:00–0:30** — “I have 200 scattered notes. Watch.”
2. **0:30–1:00** — Drag PDF, upload voice memo, paste link. They auto-appear.
3. **1:00–1:30** — Show graph: nodes colored by type, hover reveals content.
4. **1:30–2:15** — Ask: “What did I decide about X across my PDF and voice memo?” Answer cites both.
5. **2:15–2:45** — Open History, reopen past session, continue follow-up.
6. **2:45–3:00** — Show public URL. “This is SecondSelf.”

12. Deployment

- Frontend: Vercel/Netlify.
- Backend: Render/Railway/HF Spaces.
- Or single Docker container.
- Environment variables for API keys.
- Health endpoint `/health` .

13. Risks & mitigations

Risk	Mitigation
LLM rate limit	Model-agnostic fallback
OCR failure	Manual text fallback + confidence flag
Huge PDF	Chunking + async processing
Empty audio	Detect silence, mark failed
Duplicate captures	Hash raw file + text similarity
Graph too dense	Top-k links + threshold tuning
Sensitive data	Local-first, no training, redact logs

14. Final deliverables

- Public GitHub repo with README + setup.
- Live public URL.
- End-to-end flow: capture → classify → link → graph → ask → history.
- All 4 badges: Archivist, Librarian, Cartographer, Oracle.
- Clean, intentional UI — not default Streamlit.

15. Judge-facing one-liner

SecondSelf is a multi-modal personal RAG system that ingests any file type, auto-organizes it with LLMs, links it with embeddings, renders it as a live knowledge graph, and answers questions with citations while remembering every conversation.

Generated from SecondSelf PRD v3 — National Hackathon Build.